Lab 4 Report

Trajectory optimisation and drone racing

Author details to be completed

Report date: 10 October 2026

Summary

Polynomial trajectory optimisation connects waypoint geometry to the continuous state references required for quadrotor flight. This laboratory derives the single-segment minimum-velocity solutions, counts the conditions for multi-segment minimum-snap trajectories, and implements a ROS trajectory follower for the official Unity racing course. With endpoint positions alone, the optimum is $P(t) = t$ and its cost is 1 for all three polynomial degrees considered. A prescribed terminal velocity changes the solution: the cubic reduces the quadratic cost from 4 to 17/8. In the recorded Unity run, the quadrotor approached all 12 gate centres in order while tracking a 91.08 s reference trajectory. The active-flight position error had a root-mean-square (RMS) value of 0.319 m, and the final position was 0.169 m from the last gate centre.

1 Lab introduction and team information

The task is to convert a sequence of gate poses into a smooth flight reference and track it through the official simulated course. The exercises combine two analytical deliverables with a drone-racing implementation[1]. This report follows that sequence: quadratic programming explains how endpoint constraints shape a polynomial; multi-segment analysis explains how adjacent pieces join; the flight experiment shows how the resulting reference interacts with a feedback controller.

The implemented system is shown in Figure 1. Gate positions and orientations are supplied by the course map. The trajectory generator produces desired states, and the controller from Lab 3 closes the loop through the TESSE bridge. Team names, student numbers, group information and individual contributions will be added after the author details are confirmed.

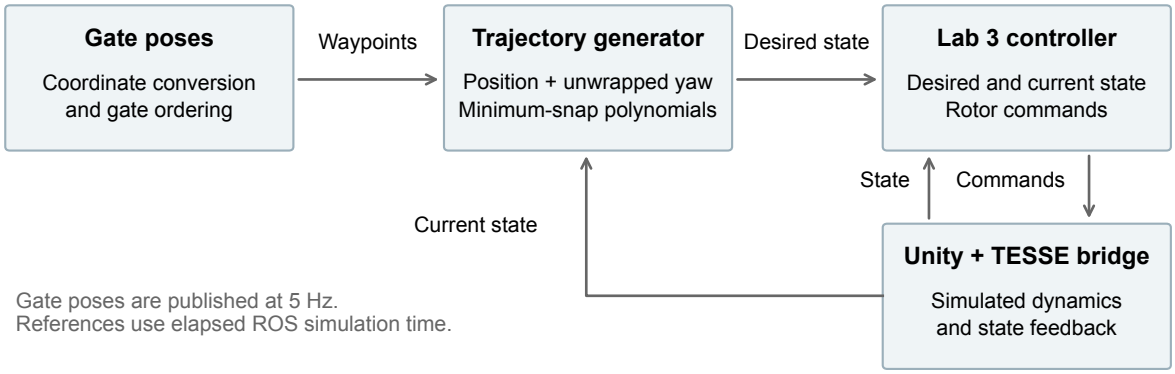


Figure 1: Implemented ROS control loop. Gate poses enter the generator on `/desired_traj_vertices`; the generator publishes `/desired_state`, and feedback arrives on `/current_state`. Arrows show command and feedback interfaces.

2 Individual work

2.1 Deliverable 1: single-segment trajectory optimisation

The objective is to minimise squared velocity on the normalised interval $[0, 1]$:

$$\min_P J(P) = \int_0^1 [P'(t)]^2 \, dt, \qquad P(0) = 0, \quad P(1) = 1. \tag{1}$$

Write $P(t) = \sum_{i=1}^{N} p_i t^i$. The constant coefficient is zero because $P(0) = 0$. With $\boldsymbol{p} = [p_1, \ldots, p_N]^T$, direct integration gives

$$J = \boldsymbol{p}^T Q \boldsymbol{p}, \qquad Q_{ij} = \frac{ij}{i + j - 1}, \qquad A\boldsymbol{p} = b, \quad A = [1, \ldots, 1], \quad b = 1. \tag{2}$$

Q is positive definite: a nonzero coefficient vector gives a nonzero derivative polynomial and hence a strictly positive integral of its square. Thus each feasible equality-constrained problem has a unique minimum. A solver that defines the objective as $\frac{1}{2}\boldsymbol{p}^T H \boldsymbol{p}$ must use $H = 2Q$.

2.1.1 Question 1: degree one

For $P(t) = p_1 t$, the endpoint equation fixes $p_1 = 1$. Therefore

$$\boxed{P^*(t) = t, \qquad J^* = 1.} \tag{3}$$

2.1.2 Question 2: degree two

For $P(t) = p_2 t^2 + p_1 t$, the requested matrices and objective are

$$Q = \begin{bmatrix} 1 & 1 \\ 1 & 4/3 \end{bmatrix}, \quad A = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad b = 1, \qquad J = p_1^2 + 2p_1 p_2 + \frac{4}{3} p_2^2. \tag{4}$$

Substituting $p_1 = 1 - p_2$ gives $J = 1 + \frac{1}{3} p_2^2$. The optimum is $\boldsymbol{p}^* = (1, 0)^T$, so $P^*(t) = t$ and $J^* = 1$. The quadratic term provides no cost reduction for these boundary conditions.

2.1.3 Question 3: degree three and the Euler–Lagrange equation

For $P(t) = p_3t^3 + p_2t^2 + p_1t$,

$$Q = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 4/3 & 3/2 \\ 1 & 3/2 & 9/5 \end{bmatrix}, \quad A = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}, \quad b = 1. \tag{5}$$

The Karush–Kuhn–Tucker (KKT) equations,

$$2Q\boldsymbol{p} + A^T\lambda = 0, \qquad A\boldsymbol{p} = b, \tag{6}$$

give $\boldsymbol{p}^* = (1, 0, 0)^T$ and $\lambda = -2$. Again $J^* = 1$. For the integrand $(P')^2$, the Euler–Lagrange equation is $P'' = 0$, which yields the same linear trajectory. Cauchy–Schwarz also gives the global bound

$$J \geq \left(\int_0^1 P'(t)\, dt \right)^2 = [P(1) - P(0)]^2 = 1. \tag{7}$$

Equality requires constant velocity, so increasing the degree cannot improve this result.

2.1.4 Question 4: prescribed terminal velocity

Adding $P'(1) = -2$ supplies a second constraint row, $[1, 2, \ldots, N]$, with $b = (1, -2)^T$. The equality-constrained minimum can be written as

$$\boldsymbol{p}^* = Q^{-1}A^T(AQ^{-1}A^T)^{-1}b. \tag{8}$$

For degree two,

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad \boldsymbol{p}^* = \begin{bmatrix} 4 \\ -3 \end{bmatrix}. \tag{9}$$

Both coefficients are fixed by the constraints, giving

$$\boxed{P^*(t) = 4t - 3t^2, \qquad J^* = \int_0^1 (4 - 6t)^2\, dt = 4.} \tag{10}$$

For degree three,

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ -2 \end{bmatrix}. \tag{11}$$

Set $p_3 = z$. The constraints give $p_1 = 4 + z$ and $p_2 = -3 - 2z$, so

$$J(z) = 4 + z + \frac{2}{15}z^2, \qquad J'(z) = 1 + \frac{4}{15}z. \tag{12}$$

Its minimum occurs at $z = -15/4$, with

$$\boxed{P^*(t) = \frac{1}{4}t + \frac{9}{2}t^2 - \frac{15}{4}t^3, \qquad J^* = \frac{17}{8}.} \tag{13}$$

The cubic satisfies both $P(1) = 1$ and $P'(1) = -2$. Its additional freedom reduces the integrated squared velocity by 46.875% relative to the quadratic. Both trajectories overshoot the final position before returning with negative velocity (Figure 2). These solutions minimise the cost within their specified polynomial spaces; the linear Euler–Lagrange solution cannot satisfy the added terminal-velocity condition.

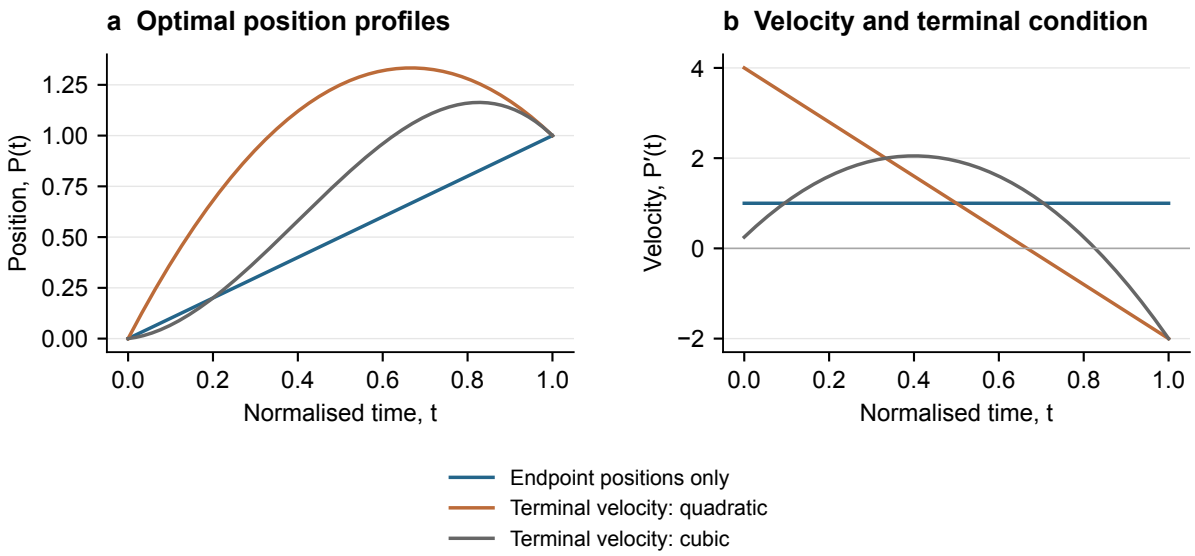


Figure 2: Exact solutions of the single-segment exercises. (a) Position profiles. (b) Their analytic derivatives. The blue line solves the position-only problem for degrees one, two and three. The other curves include $P'(1) = -2$; their costs are 4 (quadratic) and 17/8 (cubic). These are analytical curves, not flight measurements.

2.2 Deliverable 2: multi-segment trajectory optimisation

All counts below refer to one scalar coordinate, prescribed waypoint positions and fixed, strictly increasing knot times $t_0 < \cdots < t_k$. Coordinates can be treated independently when the cost is their summed squared derivative norm.

2.2.1 Question 1: minimum snap

For k segments, the snap objective is

$$J = \sum_{i=1}^{k} \int_{t_{i-1}}^{t_i} [P_i^{(4)}(t)]^2 \, dt. \tag{14}$$

Euler–Lagrange gives $P_i^{(8)} = 0$ on every open segment. Each piece therefore has degree at most seven and eight coefficients, giving $8k$ unknowns. Assume that position, velocity, acceleration and jerk are fixed at the overall start and end. The conditions determining the stationary optimum are listed in Table 1.

Table 1: Scalar condition count for k minimum-snap segments. The free-derivative row combines continuity constraints with natural optimality conditions.

Type	Conditions	Count
Waypoint	Fix both ends of each piece to the appropriate waypoint.	$2k$
Free derivative	Match derivatives of orders 1 to 6 across each interior knot.	$6(k-1)$
Fixed derivative	Specify orders 1 to 3 at the overall start and end.	6
Total	Eight conditions per polynomial piece.	$8k$

There are $k+1$ physical waypoints. Each interior waypoint supplies two position equations, one for each adjacent piece, while each overall endpoint supplies one. For $k = 3$, the counts are 6 waypoint, 12 free-derivative and 6 fixed-derivative equations, totalling 24.

Velocity, acceleration and jerk continuity define admissible trajectories. Their common values at interior knots remain free. Minimising snap then supplies continuity of derivatives four, five and six as natural optimality conditions. The seventh derivative may jump because the interior positions are prescribed. Thus a QP needs $2k + 3(k-1) + 6 = 5k + 3$ explicit equality constraints; optimisation supplies the remaining $3(k-1)$ conditions. Treating all $8k$ equations as prescribed derivative values would remove the distinction between feasibility and optimality.

2.2.2 Question 2: a general derivative order

For an objective involving the squared rth derivative,

$$J = \sum_{i=1}^{k} \int_{t_{i-1}}^{t_i} [P_i^{(r)}(t)]^2 \, dt, \tag{15}$$

Euler–Lagrange gives $P_i^{(2r)} = 0$. The degree is at most $2r-1$, with $2rk$ coefficients in total. Prescribe endpoint derivatives of orders $1, \ldots, r-1$. The counts become

$$\boxed{\underbrace{2k}_{\text{waypoint}} + \underbrace{(2r-2)(k-1)}_{\text{free derivative}} + \underbrace{2(r-1)}_{\text{fixed derivative}} = 2rk.} \tag{16}$$

The free-derivative equations match orders $1, \ldots, 2r-2$ at each interior knot. Orders $1, \ldots, r-1$ are continuity constraints; orders $r, \ldots, 2r-2$ follow from stationarity. The explicit QP has $(r+1)k + r - 1$ equality constraints and $(r-1)(k-1)$ free directions.

To obtain the natural conditions, integrate the first variation by parts r times:

$$\frac{1}{2}\delta J_i = \left[\sum_{j=0}^{r-1} (-1)^j P_i^{(r+j)} \delta P_i^{(r-1-j)} \right]_{t_{i-1}}^{t_i} + (-1)^r \int_{t_{i-1}}^{t_i} P_i^{(2r)} \delta P_i \, dt. \tag{17}$$

At an interior waypoint, $\delta P = 0$. The remaining common variations $\delta P^{(1)}, \ldots, \delta P^{(r-1)}$ are arbitrary, so their boundary terms require continuity of orders $r, \ldots, 2r-2$. The count also covers $k = 1$ and $r = 1$: the corresponding interior or derivative conditions disappear.

3 Team work

3.1 Deliverable 3: trajectory generation and following

The implementation uses the provided polynomial optimisation library[2] and completes Part 0 and Parts 1.1–1.3 of the exercise. The course contains 12 ordered gate poses. The position conversion from Unity coordinates to the ROS world frame is

$$(x_R, y_R, z_R) = (x_U, z_U, y_U). \tag{18}$$

For quaternion components ordered as (x, y, z, w), the corresponding conversion is $(q_{Rx}, q_{Ry}, q_{Rz}, q_{Rw}) = (-q_{Ux}, -q_{Uz}, -q_{Uy}, q_{Uw})$. The waypoint publisher sorts gates by their numerical index and publishes a `PoseArray` at 5 Hz.

3.1.1 Part 0: first-gate reference

The simple planner takes the first gate pose, normalises its quaternion and publishes a desired state with zero linear and angular velocity and acceleration. A 0.01 s timer keeps the command active and broadcasts the desired pose in the world frame. Empty lists and invalid first poses are rejected. The saved hover record contains the position $(0.423, 3.165, 2.374)$ m; this individual sample is not used to estimate a settling time or a steady-state error statistic.

3.1.2 Part 1.1: initialise from the current state

The follower obtains position and yaw from `/current_state` and waits for a valid state before planning. Waypoints received first are retained until that state arrives. The initial vertex is the measured position, which joins the current vehicle location to the first gate. Once a trajectory exists, repeated waypoint messages do not restart it.

3.1.3 Part 1.2: optimise position and yaw

The generator builds a three-dimensional position trajectory and a scalar yaw trajectory using identical segment durations. Interior vertices constrain the gate position and yaw. Successive yaw angles are unwrapped using

$$\psi_i = \psi_{i-1} + \operatorname{atan2}\Big(\sin(\bar{\psi}_i - \psi_{i-1}), \cos(\bar{\psi}_i - \psi_{i-1})\Big), \tag{19}$$

where $\bar{\psi}_i$ is the yaw extracted from the gate quaternion. This selects the short angular increment across the $\pm\pi$ boundary.

Both optimisers minimise snap using ten coefficients per segment, so their polynomials have degree at most nine. The library shares endpoint derivatives through order four at interior vertices. Calls to `makeStartOrEnd(..., SNAP)` set derivatives one through four to zero at the overall endpoints. This implementation therefore includes endpoint snap constraints and uses a different polynomial space from the minimal degree-seven derivation in Deliverable 2. The mathematical count there applies to the stated endpoint-through-jerk problem.

The segment-time estimator uses gate spacing with parameters $v_{\max} = 2\,\mathrm{m\,s^{-1}}$ and $a_{\max} = 1\,\mathrm{m\,s^{-2}}$. A time scale of 1 is applied and every segment is at least 0.5 s long. These parameters choose durations; the linear QP does not impose them as hard velocity or acceleration bounds.

3.1.4 Part 1.3: publish the desired state

The generator evaluates position, velocity, acceleration, yaw, yaw rate and yaw acceleration at the elapsed ROS time. The sample time is clipped to $[0, T]$, where T is the trajectory duration. The final rest reference therefore continues after T. The desired-state timer is configured at 100 Hz; the controller timer is configured at 200 Hz. These settings describe requested timer rates, not measured throughput of the simulator.

3.2 Experimental configuration and measurement

The recorded experiment used the official Lab 4 Unity 2019.3.9f1 simulator with Box64 0.3.6 on ARM64 and ROS Noetic. The selected controller gains were $k_x = 1.2$, $k_v = 2.0$, $k_R = 1.2$ and $k_\omega = 0.7$, with drag compensation disabled. State and desired-state timeout settings were both 1 s. The saved build log reports seven successfully built packages, and the saved integration checks cover input validation, initial-state waiting, repeated waypoint messages, yaw wrapping, derivative consistency and endpoint holding.

Results below use one recorded Unity run. The telemetry contains 6,064 rows; each row pairs a current-state position with the latest received desired-state position. This is the recorded online pairing, without subsequent temporal interpolation. For sample j, the Euclidean tracking error is

$$e_j = \|\boldsymbol{x}_j - \boldsymbol{x}_{d,j}\|_2, \qquad e_{\mathrm{RMS}} = \sqrt{\frac{1}{n}\sum_{j\in\mathcal{A}} e_j^2}, \tag{20}$$

where $\mathcal{A}$ contains the 4,556 samples whose reference time is below $T - 0.01\,\mathrm{s}$ and $n = |\mathcal{A}|$. The remaining 1,508 samples cover the terminal reference hold. This distinction keeps the active-flight metric separate from the terminal behaviour. All samples are retained in the time-series figures.

For gate i, the closest recorded centre distance and its time are

$$d_i = \min_j \|\boldsymbol{x}_j - \boldsymbol{g}_i\|_2, \qquad t_i^{\mathrm{near}} = t_{\arg\min_j \|\boldsymbol{x}_j - \boldsymbol{g}_i\|_2}. \tag{21}$$

Here $\boldsymbol{g}_i$ is the gate centre in ROS world coordinates. These are closest-approach times in the complete recording, including the final hold. The statistics describe 6,064 time samples within one run.

3.3 Recorded flight through the course

The trajectory and measured flight follow the sequence of all 12 gate centres (Figure 3). The reference lasts 91.0797 s, while the full telemetry record spans 121.30 s. The terminal reference is first recorded at 91.16 s on the telemetry clock and remains active for the remaining 30.14 s.

Table 2: Measurements from the single recorded Unity run. Tracking statistics use active-flight samples; final position error uses the last sample.

Quantity	Value
Reference trajectory duration	91.0797 s
Total telemetry duration	121.30 s
Active-flight RMS position error	0.319 m
Active-flight maximum position error	0.578 m
Largest closest-approach distance among the gates	0.448 m
Final position error relative to the last gate centre	0.169 m
Terminal-hold minimum / maximum position error	0.151 / 0.585 m

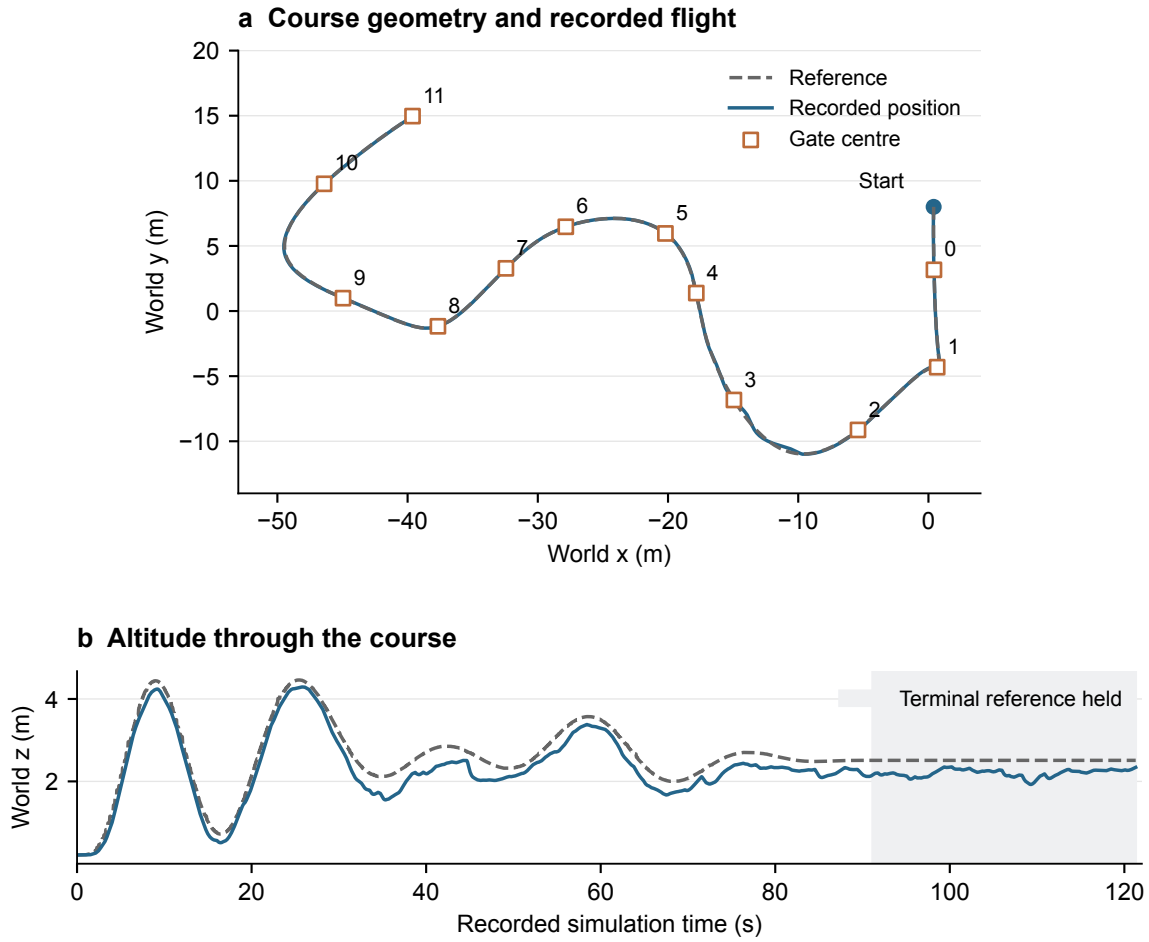


Figure 3: Official Unity flight from the recorded telemetry. (a) Horizontal projection of the reference and measured trajectory, with gate indices 0–11. (b) Reference and measured altitude over the full recording. The shaded region marks the terminal reference hold. All 6,064 samples are plotted; equal scaling of the horizontal axes preserves course geometry.

3.4 Tracking error and gate-centre visits

The active-flight RMS error was 0.319 m, with a maximum of 0.578 m. Figure 4 shows the complete error trace and the closest distance to every gate. The closest-approach times increase strictly with gate index (Table 3), and each closest distance is below 0.45 m. This 0.45 m value describes the observed distances; it is not a course-defined acceptance threshold. Gates 4 and 5 have the largest closest distances, 0.445 and 0.448 m.

3.5 Simulator view and terminal behaviour

The retained simulator screenshot in Figure 5 places the telemetry in its visual context. It shows the quadrotor near a green gate with additional gates visible along the course. The saved screen recording and rosbag accompany this report locally. The bag metadata records 6,284 current-state messages and 3,125 desired-state messages over approximately 125 s; its recording starts before the paired telemetry series, so the durations and sample counts differ.

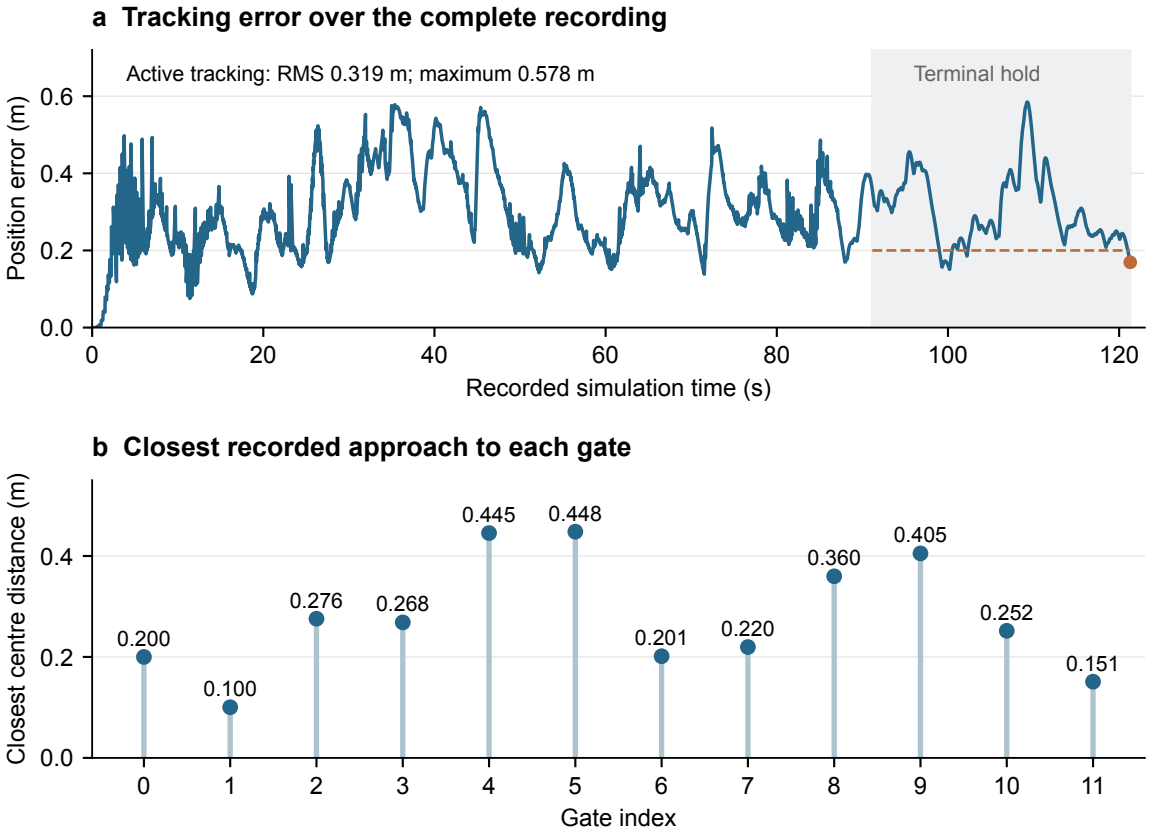


Figure 4: Tracking accuracy and gate-centre proximity. (a) Euclidean error for all recorded samples. The shaded interval is the terminal hold; the dashed 0.20 m line is the local offline terminal tolerance, and the orange point is the last sample. (b) Minimum Euclidean distance to each gate centre over the complete recording. All measurements are from one Unity flight.

Table 3: Closest-approach times and distances. Gate numbering follows the course map. The final gate's closest approach occurs during the terminal hold.

Gate	Time (s)	Distance (m)	Gate	Time (s)	Distance (m)
0	5.72	0.200	6	52.94	0.201
1	13.02	0.100	7	58.82	0.220
2	21.18	0.276	8	65.86	0.360
3	30.82	0.268	9	73.64	0.405
4	39.66	0.445	10	82.46	0.252
5	44.88	0.448	11	100.18	0.151

The last position was 0.169 m from the final gate centre. Over the terminal hold, the error ranged from 0.151 to 0.585 m, so the final sample satisfies the offline 0.20 m tolerance without establishing that the entire hold stayed inside it. The original live-monitor log ended with a failed completion check; the accompanying run notes identify its threshold as 0.08 m. The later offline check uses 0.20 m. Reporting the trace and final distance preserves the measured outcome independently of either threshold.

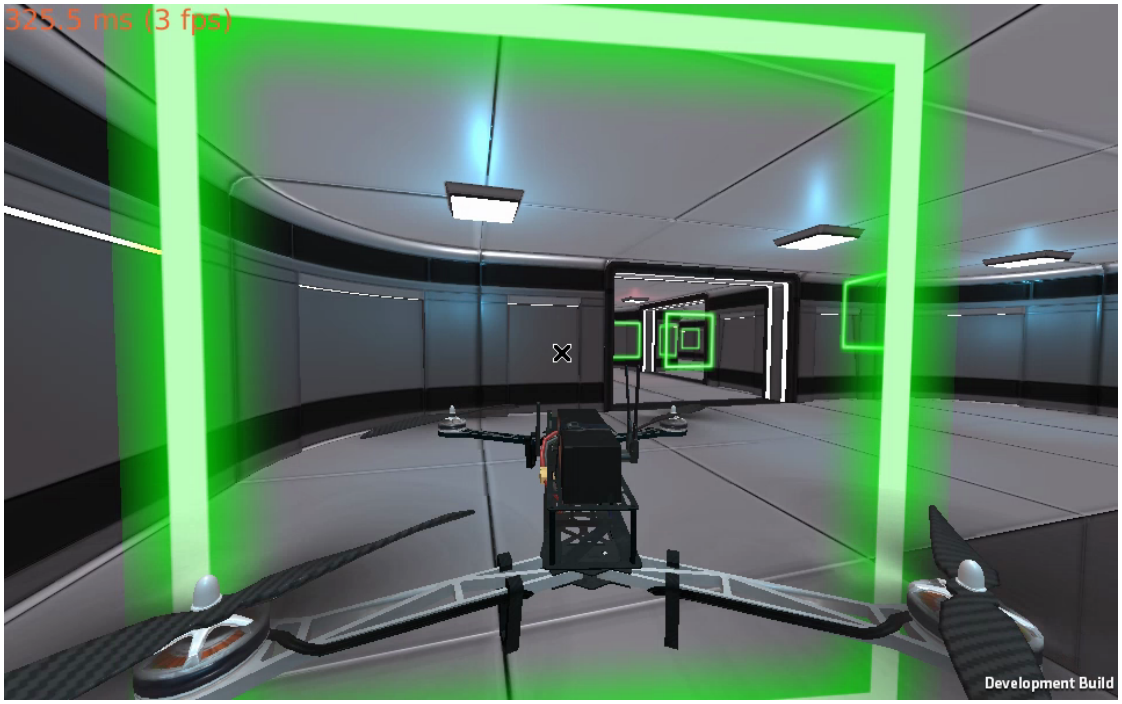


Figure 5: Retained view from the official Unity experiment. The screenshot is reproduced in full, with its original colours and aspect ratio. It supplies scene context for the trajectory plots; the gate-centre distances come from telemetry.

4 Discussion

4.1 Polynomial freedom and boundary conditions

The single-segment solutions show when a higher polynomial degree is useful. With position constraints alone, constant velocity already reaches the global lower bound. The terminal-velocity condition changes the feasible set: a cubic can redistribute velocity over the interval and reduce the cost from 4 to 17/8. The overshoot in Figure 2 follows from arriving at the final point with negative velocity. Boundary conditions therefore determine the value of extra coefficients more directly than polynomial degree alone.

For multiple segments, the distinction between explicit constraints and natural conditions explains how the optimiser chooses free waypoint derivatives. In the flight implementation, shared position and yaw segment times keep the reference components synchronised, while angle unwrapping avoids a full-turn interpolation at a representation discontinuity. Endpoint sampling then gives the controller a persistent rest reference after the planned motion ends.

4.2 What the flight measurements establish

The recorded run supports ordered gate-centre visits and quantifies tracking over the course. Centre proximity is a spatial diagnostic: a collision-free crossing of every gate would require crossing geometry or simulator event records in addition to these distances. Likewise, 91.08 s is the reference duration, not an independently measured official race time. The optional speed challenge is outside the demonstrated result.

The terminal trace shows continued position variation after the reference stops. A time-window settling criterion would be needed to quantify sustained hover accuracy. No controller comparison is available to attribute that variation to a particular gain, bridge timing effect or simulator model. The present results therefore describe the selected configuration and recorded run. They provide a concrete starting point for any subsequent time-allocation or gain study.

5 Conclusions

The analytical exercises connect polynomial coefficients to boundary conditions and identify the continuity and stationarity conditions for multi-segment planning. The implemented ROS follower extends that formulation to spatial waypoints and unwrapped yaw, then supplies the derivatives needed by the Lab 3 controller. In the official Unity recording, the vehicle approached all 12 gate centres in order with an active-flight RMS position error of 0.319 m. Together, the derivations and recorded flight demonstrate the complete chain from waypoint constraints to a tracked reference trajectory.

References

[1] MIT Visual Navigation for Autonomous Vehicles. *Lab 4: Exercises*, 2023 course materials. `https://vnav.mit.edu/labs_2023/lab4/exercises.html`. Accessed 10 October 2026.

[2] ETH Zurich Autonomous Systems Lab. *mav_trajectory_generation: Polynomial trajectory generation and optimisation*. `https://github.com/ethz-asl/mav_trajectory_generation`. Library documentation and the implementation distributed with the laboratory. Accessed 10 October 2026.